CPSC 368
Databases in Data Science

Structured Query Language (SQL)

Textbook Reference
Database Management Systems: Chapter 5

Learning Goals

- Define the components (and synonyms) of the relational model: tables, rows, columns, keys, associations, etc.
- Create tables, including the attributes, keys, and field lengths, using Data Definition Language (DDL)
- Given the schemas of a relation, create SQL queries using: SELECT, FROM, WHERE, EXISTS, NOT EXISTS, UNIQUE, NOT UNIQUE, ANY, ALL, DISTINCT, GROUP BY and HAVING.
- Show that there are alternative ways of coding SQL queries to yield the same result. Determine whether or not two SQL queries are equivalent.
- Given a SQL query and table schemas and instances, compute the query result.
- Translate a query between SQL and English.
- Explain the purpose of NULL values and justify their use. Also describe the difficulties added by having nulls.
- Create and modify table schemas and views in SQL.
- Explain the role and advantages of embedding SQL in application programs.
- Write SQL for a small-to-medium sized programming application that requires database access.
- Identify the pros and cons of using general table constraints (e.g., CONSTRAINT, CHECK) and triggers in databases.

Databases: The Continuing Saga

We motivated that databases were great because they:

- Store large amounts of data
- Handle transactions
- Allow efficient querying
- And many, many more classic favourites!

Before we can do all of these, we must design the database

Example of a Relation Instance

relation name

attribute, column name

tuple, row, record

domain value

Student

sid	name	address	phone	major
99111120	K. Jones	1234 W. 12th Ave., Van	889-4444	CPSC
92001200	S. Selvarajah	2020 E. 18th St., Van	409-2222	MATH
94001020	A. Alberty	2020 E. 18th St., Van	222-2222	FREN
94001150	J. Wang	null	null	null

- degree/arity = 5; Cardinality = 4,
- Order of rows isn't important
- Order of attributes isn't important (except in some query languages)

Structure of Relational Databases

- ***Relational database**:* a set of ***relations***
- ***Relation:*** made up of 2 parts:
  - ***Schema*** : specifies name of relation, plus name and **domain** (type) of each **attribute**.
    - e.g., Student (*sid*: string, *name*: string, *address*: string, *phone*: string, *major*: string).
  - ***Instance*** : a ***table***, with rows and columns.
    #Rows = cardinality
    #Columns = arity / degree
- ***Relational Database Schema:*** collection of schemas in the database
- ***Database Instance:*** a collection of instances of its relations

Formal Structure

- Formally, a relation r is a set $(a_1, a_2,\ldots,a_n)$ where a_i is in D_i , the domain (set of allowed values) of the i-th attribute.
- Attribute values are atomic, i.e., integers, floats, strings
- A domain contains a special value ***null*** indicating that the value is not known.
- If $A_1,\ldots, A_n$ are attributes with domains $D_1,\ldots D_n$, then
  $(A_1{:}D_1, \ldots, A_n{:}D_n)$
  is a ***relation schema*** that defines a relation type – sometimes we leave off the domains

Example of a formal definition

Student

sid	name	address	phone	major
99111120	K. Jones	1234 W. 12th Ave., Van	889-4444	CPSC
92001200	S. Selvarajah	2020 E. 18th St., Van	409-2222	MATH
94001020	A. Alberty	2020 E. 18th St., Van	222-2222	FREN
94001150	J. Wang	null	null	null

Student(sid: integer, name: string, address: string, phone: string, major: string)

Or without the domains:

Student (sid, name, address, phone, major)

Clicker Question

Here is a table representing a relation named R. Identify the attributes,
schema, and tuples of R.

A	B	C
0	1	2
3	4	5
6	7	8
9	10	11

Which of the following is NOT a true statement about R?

A.R has four tuples.

B.B is an attribute of R.

C.(6,7,8) is a tuple of R.

D.The schema of R is R(A,B,C).

E.None of the above

Creating Relations in SQL/DDL

- The statement on the right creates the Student relation
  - the type (domain) of each attribute is specified and enforced when tuples are added or modified

```
CREATE TABLE Student
        (sid        INTEGER,
         name       CHAR(20),
         address    CHAR(30),
         phone      CHAR(13),
         major      CHAR(4))
```

sid	name	address	phone	major

Domain Types in SQL Reference Sheet

- **char(n).** Fixed length character string with length *n.*
- **varchar(n).** Variable length character strings, with maximum length *n.*
- **int.** Integer (machine-dependent).
- **smallint.** Small integer (machine-dependent).
- **numeric(p,d).** Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point.
- **real, double precision.** Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.

- Null values are allowed in all the domain types.
  To prohibit null values declare attribute to be **not null**
- **create domain** in SQL-92 and 99 creates user-defined domain types

  create domain *person-name* **char**(20) **not null**

Date/Time Types in SQL Reference Sheet

- **date.** Dates, containing a (4 digit) year, month and date
  - E.g. **date** '2001-7-27'
- **time.** Time of day, in hours, minutes and seconds.
  - E.g. **time** '09:00:30' **time** '09:00:30.75'
- **timestamp**: date plus time of day
  - E.g. **timestamp** '2001-7-27 09:00:30.75'
- **Interval**: period of time
  - E.g. Interval '1' day
  - Subtracting a date/time/timestamp value from another gives an interval value
  - Interval values can be added to date/time/timestamp values
- Relational DBMS offer a variety of functions to
  - extract values of individual fields from date/time/timestamp
  - convert strings to dates and vice versa
  - For instance in Oracle (date is a timestamp):
    - TO_CHAR(date, format)
    - TO_DATE(string, format)
    - format looks like: 'DD-Mon-YY HH:MI.SS'

Destroying and Altering Relations

DROP TABLE Student

- Destroys the relation Student. Schema information *and* tuples are deleted.

ALTER TABLE Student
ADD COLUMN gpa REAL;

- The schema of Students is altered by adding a new attribute; every tuple in current instance is extended with a ***null*** value in the new attribute.

Adding and Deleting Tuples

- Can insert a single tuple using:

```
INSERT
INTO      Student (sid, name, address, phone, major)
VALUES  (52033688, 'G. Chan', '1235 W. 33, Van',
          '882-4444',  'PHYS')
```

- Can delete all tuples satisfying some condition (e.g., name = 'Smith'):

```
DELETE
FROM    Student
WHERE   name = 'Smith'
```

Powerful variants of these commands exist; more later

Keys Constraints (for Relations)

- One or more attributes in a relation form a ***key*** (or ***candidate key***) for a relation, where S is the set of all attributes in the key, if:
  1. No distinct tuples can have the same values for all attributes in the key, and
  2. No subset of S is itself a key (according to (1)).

  (If such a subset exists, then S is a *superkey* and not a key.)
- One of the possible keys is chosen (by the DBA) to be the ***primary key*** (PK).

```
CREATE TABLE Student
        (sid      INTEGER  PRIMARY KEY,
         name     CHAR(20),
         address  CHAR(30),
         phone    CHAR(13),
         major    CHAR(4))
```

Keys Constraints in SQL

- A **PRIMARY KEY** constraint specifies a table's primary key
  - values for primary key must be unique
  - a primary key attribute cannot be *null*
- Other keys are specified using the **UNIQUE** constraint
  - values for a group of attributes must be unique (if they are not null)
  - these attributes can be *null*
- Key constraints are checked when
  - new values are inserted
  - values are modified

```
CREATE TABLE Student (
    sid INTEGER,
    name CHAR(20),
    address  CHAR(30),
    phone    CHAR(13),
    major    CHAR(4),
    PRIMARY KEY (sid)
)
```

Clicker Question

Does the constraint PRIMARY KEY(Dept, Course#) hold for this instance?

Course#	Dept	Title
100	CPSC	Computational Thinking
100	MATH	Differential Calculus with Applications to Physical Sciences and Engineering

A. Yes

B. No

C. It depends

Clicker Question

Does the constraint PRIMARY KEY(Dept, Course#) hold for this instance?

Dept	Course #	Term	Section
CPSC	368	2021W2	201
CPSC	368	2021W2	202

A. Yes
B. No
C. It depends

Keys Constraints in SQL (cont')

For single attribute keys, can also be declared on the same line as the attribute.

```
CREATE TABLE Student
        (sid        INTEGER  PRIMARY KEY,
         name       CHAR(20),
         address    CHAR(30),
         phone      CHAR(13),
         major      CHAR(4))
```

Keys Constraints in SQL (cont')

(Ex.1- Normal) "For a given student and course, there is a single grade."

```
CREATE TABLE Grade
  (sid        INTEGER,
   dept       CHAR(4),
   course#    CHAR(3),
   mark       INTEGER,
   PRIMARY KEY (sid,dept,course#) )
```

vs.

(Ex.2 - Silly) "Students can take a course once, and receive a single grade for that course; further, no two students in a course receive the same grade."

```
CREATE TABLE Grade2
  (sid        INTEGER,
   dept       CHAR(4),
   course#    CHAR(3),
   mark       CHAR(2),
   PRIMARY KEY (sid,dept,course#),
   UNIQUE (dept,course#,mark) )
```

Students and Grades

Student

sid	name	…
1	Blossom	…
2	Buttercup	…
3	Bubbles	…
4	Blossom	

Grade

sid	dept	cnum	grade
2	CPSC	304	90
2	MATH	221	90
2	EPSE	223	90
1	MUSC	103	90

What does the sid attribute in Grade refer to?

Enforcing Referential Integrity

Student

sid	name	…
1	Blossom	…
2	Buttercup	…
3	Bubbles	…
4	Blossom	

Grade

sid	dept	cnum	grade
2	CPSC	304	90
2	MATH	221	90
2	EPSE	223	90
1	MUSC	103	90

A **foreign key** is a set of attributes in one relation (e.g., Grades.sid) used to 'reference' a tuple in another relation (e.g., Students.sid).

- **Must correspond to the primary key of the other relation.**
- Like a 'logical pointer'.

Enforcing Referential Integrity

Grade

sid	dept	cnum	grade
2	CPSC	304	90
2	MATH	221	90
2	EPSE	223	90
1	MUSC	103	90

```
CREATE TABLE Grade (
        sid INTEGER,
        dept  CHAR(4),
        course# CHAR(3),
        mark INTEGER,
        PRIMARY KEY  (sid,dept,course#),
        FOREIGN KEY (sid) REFERENCES
                Student,
        FOREIGN KEY (dept, course#)
                REFERENCES Course(dept, cnum)
    )
```

Let's look at students and grades again...

Student

sid	name	...
1	Blossom	...
2	Buttercup	...
3	Bubbles	...
4	Blossom	

Grade

sid	dept	cnum	grade
2	CPSC	304	90
2	MATH	221	90
2	EPSE	223	90
1	MUSC	103	90

Do we want tuples in Grade to ever refer to a sid that does not exist?

NO!

Idea: Check the Student table each time we insert a tuple into the Grade table to see if sid exists.

Enforcing Referential Integrity

- **Referential integrity**: All foreign keys reference existing entities.

- *sid* in Grade is a foreign key that references Student.
- What should be done if a Grade tuple with a non-existent student id is inserted? (*Reject it!*)
- What should be done if a **Student tuple** is deleted?
  - Also delete all Grade tuples that refer to it?
  - Disallow deletion of this particular Student tuple?
  - Set sid in Grade tuples that refer to it, to *null*, (the special value denoting *`unknown'* or *`inapplicable'*.)
    - Problem if sid is part of the primary key
  - Set sid in Grade tuples that refer to it, to a *default sid*.
- Similar if primary key of a Student tuple is updated

Referential Integrity in SQL/92

- SQL/92 supports all 4 options on deletes and updates.
  - Default is **NO ACTION** (*delete/update is rejected*)
  - **CASCADE** (also updates/deletes all tuples that refer to the updated/deleted tuple)
  - **SET NULL / SET DEFAULT** (referencing tuple value is set to the default foreign key value)

```
CREATE TABLE Grade(
   sid CHAR(8),
   dept  CHAR(4),
   course# CHAR(3),
   mark INTEGER,
   PRIMARY KEY  (sid,dept,course#),
   FOREIGN KEY (sid) REFERENCES
     Student(sid)
          ON DELETE CASCADE
          ON UPDATE CASCADE
   FOREIGN KEY (dept, course#)
      REFERENCES Course(dept,course#)
          ON DELETE SET DEFAULT
          ON UPDATE CASCADE );
```

Clicker Question

Consider the following table definition and relational instances.

```
CREATE TABLE  BMW  ( bid  INTEGER, sid INTEGER,  …
            PRIMARY KEY (bid),
            FOREIGN KEY (sid) REFERENCES STUDENTS
                ON DELETE CASCADE);
```

BMW

bid	Sid
1000	5678

Student

sid	name	Address
5678	James	Null

Based on the SQL DDL statement above, which statement is true?

A. The sid value in a Student tuple depends on an existing BMW tuple
B. The sid value in a BMW tuple depends on an existing Student tuple
C. The tuples in BMW and Student have no relation to each other

Clicker Question

Consider the following table definition and relational instances.

```
CREATE TABLE  BMW  ( bid  INTEGER, sid INTEGER,  …
            PRIMARY KEY (bid),
            FOREIGN KEY (sid) REFERENCES STUDENTS
                ON DELETE CASCADE);
```

BMW

bid	Sid
1000	5678

Student

sid	name	Address
5678	James	Null

A. If the row for sid value 5678 in STUDENTS is deleted, then the row with bid = 1000 in BMW is automatically deleted.

B. If a row with sid value 5678 in BMW is deleted, then the row with sid=5678 in STUDENTS is automatically deleted.

C. Both of the above.

Coming up in SQL…

- Basic Structure
- Set Operations
- Aggregate Functions
- Null Values
- Nested Subqueries
- Modification of the Database
- Views
- Integrity Constraints
- Putting SQL to work in an application

The SQL Query Language

- Need for a standard since relational queries are used by many vendors
- Consists of several parts:
  - Data Definition Language (DDL)
  - Data Manipulation Language (DML)
    - Data Query
    - Data Modification

Running Example

Movie(<u>MovieID</u>, Title, Year)

StarsIn(<u>MovieID, StarID</u>, Character)

MovieStar(<u>StarID</u>, Name, Gender)

Basic SQL Query

- SQL is based on set and relational operations
- A typical SQL query has the form:

  SELECT $A_1, A_2, \ldots, A_n$
  FROM $r_1, r_2, \ldots, r_m$
  WHERE P

  - A_is represent attributes
  - r_is represent relations
  - P is a predicate.

SELECT	*target-list*
FROM	*relation-list*
WHERE	*qualification*

- The result of a SQL query is a table (relation)
- By default, duplicates are not eliminated in SQL relations, which are bags or multisets and not sets

Basic SQL Query

SELECT $A_1, A_2, \ldots, A_n$
FROM $r_1, r_2, \ldots, r_m$

SELECT	*target-list*
FROM	*relation-list*

- This is where we specify which column(s) we want in our answer
- If you want all the columns, use * (`SELECT *`)

Basic SQL Query

SELECT $A_1, A_2, \ldots, A_n$
FROM $r_1, r_2, \ldots, r_m$

SELECT	*target-list*
FROM	*relation-list*

- This is where we specify which table(s) you want to query
- Separate each table with a comma

Clicker Question: SQL projection

What is the result of:

```
SELECT Score1,Score2
FROM Scores
```

Scores

Team1	Team2	Score1	Score2
Dragons	Tigers	5	3
Carp	Swallows	4	6
Bay Stars	Giants	2	1
Marines	Hawks	5	3
Ham Fighters	Buffaloes	1	6
Lions	Golden Eagles	8	12

Which of the following rows is in the answer?

A. (1,2)
B. (5,3)
C. (8,6)
D. All are in the answer
E. None are in the answer

Basic SQL Query

SELECT $A_1, A_2, \ldots, A_n$
FROM $r_1, r_2, \ldots, r_m$
WHERE P

SELECT	*target-list*
FROM	*relation-list*
WHERE	*qualification*

- What conditions do the tuples in your result have to adhere to?
  - ALL tuples in the result must meet the conditions listed in the WHERE
- Another way to think about it: what conditions will you use to rule out the tuples you don't want?

WHERE Clause

```
SELECT  *
FROM    Movie
WHERE   Year > 1939
```

You can use:

- attribute names of the relation(s) used in the FROM.
- comparison operators: =, <>, <, >, <=, >=
- apply arithmetic operations: rating*2
- operations on strings (e.g., "||" for concatenation).
- Lexicographic order on strings.
    - Pattern matching: s LIKE p
    - Special stuff for comparing dates and times.

Clicker Question: Selection

```
SELECT *
FROM Scores
WHERE RunsFor > 5
```

Which tuple is in the result?

A. (Swallows, Carp, 6, 4)
B. (Swallows, Carp, 4)
C. (12)
D. (*)

Scores

Team	Opponent	Runs For	Runs Against
Dragons	Tigers	5	3
Carp	Swallows	4	6
Bay Stars	Giants	2	1
Marines	Hawks	5	3
Ham Fighters	Buffaloes	1	6
Lions	Golden Eagles	8	12
Tigers	Dragons	3	5
Swallows	Carp	6	4
Giants	Bay Stars	1	2
Hawks	Marines	3	5
Buffaloes	Ham Fighters	6	1
Golden Eagles	Lions	12	8

SELECT, FROM, WHERE

We can put these together:

- What are the names of female movie stars?

- What are the titles of movies from prior to 1939?

Movie(<u>MovieID</u>, Title, Year)
StarsIn(<u>MovieID, StarID</u>, Character)
MovieStar(<u>StarID</u>, Name, Gender)

Selection Example (Dates)

reserves

SID	BID	Day
22	101	2010-10-10
22	102	2010-10-10
22	103	2010-10-08
22	104	2010-07-10
31	102	2010-11-10
31	103	2010-11-06
31	104	2010-11-12
58	102	2010-11-08
58	103	2010-11-12

```
SELECT *
FROM reserves
WHERE day < DATE'2010-11-01'
```

SID	BID	Day
22	101	2010-10-10
22	102	2010-10-10
22	103	2010-10-08
22	104	2010-07-10

Joins in SQL

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

```
SELECT   Character, Name
FROM     StarsIn s, MovieStar m
WHERE    s.StarID = m.StarID
```

- Can alias relations (e.g., "StarsIn s")
- Conditions specified in WHERE clause
- Joins are cross products

Joins in SQL

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

```
SELECT   Character, Name
FROM     StarsIn s, MovieStar m
WHERE    s.StarID = m.StarID
```

- Can alias relations (e.g., "StarsIn s")
- Conditions specified in WHERE clause
- Joins are cross products

Cross Product (x)

- Produces every possible combination of tuples

R

A	B
0	0
0	1
1	0
1	1

S

C	D
0	0
0	1

R x S

A	B	C	D
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	0
0	0	0	1
0	1	0	1
1	0	0	1
1	1	0	1

Clicker Question: Simple Joins

```
SELECT R.a, R.b, S.a, S.b
FROM R, S
WHERE R.b = S.a
```

R

a	b
0	0
0	1
1	0
1	1

S

a	b
0	0
0	1
1	0
1	1

Which statement is true?

A. (0,1,1,0) appears twice.

B. (1,1,0,1) appears once.

C. (1,1,1,0) appears once.

D. All are true

E. None are true

Clicker Question: Joins

```
SELECT R.a, R.b, S.b, T.b
  FROM R, S, T
  WHERE R.b = S.a AND S.b <> T.b
```

(note: <> == 'not equals')

Which of the following is true:

A. (0,1,1,0) appears twice.
B. (1,1,0,1) does not appear.
C. (1,1,1,0) appears once.
D. All are true
E. None are true

R

a	b
0	0
0	1
1	0
1	1

S

a	b
0	0
0	1
1	0
1	1

T

a	b
0	0
0	1
1	0
1	1

Using DISTINCT

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

- Find the names of actors who have been in at least one movie

```
SELECT DISTINCT Name
FROM StarsIn S, MovieStar M
WHERE S.StarID = M.StarID
```

- Would removing DISTINCT from this query make a difference?
- Note: on the exams, if we ask for a general question like "find all the names", **we expect duplicates to be removed**.

Clicker Question

Consider the relation Scores(Team, Opponent, RunsFor, RunsAgainst) and the query:

```
SELECT DISTINCT Team,
                RunsFor
FROM Scores
```

Which is true:

A. 1 appears once
B. 5 appears twice
C. 6 appears 4 times
D. All are true
E. None are true

Team	Opponent	Runs For	Runs Against
Dragons	Tigers	5	3
Carp	Swallows	4	6
Bay Stars	Giants	2	1
Marines	Hawks	5	3
Ham Fighters	Buffaloes	1	6
Lions	Golden Eagles	8	12
Tigers	Dragons	3	5
Swallows	Carp	6	4
Giants	Bay Stars	1	2
Hawks	Marines	3	5
Buffaloes	Ham Fighters	6	1
Golden Eagles	Lions	12	8

Join Example

- Find the names of all movie stars who have been in a movie

Join Example

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

- Find the names of all movie stars who have been in a movie

```
SELECT Name
FROM StarsIn S, MovieStar M
WHERE S.StarID = M.StarID
```

Is this totally correct?

StarID	Name	Gender
1	Harrison Ford	Male
2	Vivian Leigh	Female
3	Judy Garland	Female

MovieID	StarID	Character
1	1	Han Solo
4	1	Indiana Jones
2	2	Scarlett O'Hara
3	3	Dorothy Gale

Harrison Ford will appear twice

Join Example

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

- Find the names of all movie stars who have been in a movie

```
SELECT Name
FROM StarsIn S, MovieStar M
WHERE S.StarID = M.StarID
```

Is this totally correct?

- What if I run the following query?

```
SELECT DISTINCT Name
FROM StarsIn S, MovieStar M
WHERE S.StarID = M.StarID
```

What if two movie stars had the same name?

```
SELECT DISTINCT StarID, Name
FROM StarsIn S, MovieStar M
WHERE S.StarID = M.StarID
```

Error: Column StarID is ambiguous

Select Project Join Example

What are all the titles of movies with female actors?

Movie(<u>MovieID</u>, Title, Year)
StarsIn(<u>MovieID, StarID</u>, Character)
MovieStar(<u>StarID</u>, Name, Gender)

Renaming Attributes in Result

- SQL allows renaming relations and attributes using the **as** clause:

  old-name **as** *new-name*

- Example: Find the title of movies and the IDs of all actors in them, and rename "StarID" to "ID"

```
SELECT Title, StarID AS ID
FROM StarsIn S, Movie M
WHERE M.MovieID = S.MovieID
```

Movie(<u>MovieID</u>, Title, Year)
StarsIn(<u>MovieID, StarID</u>, Character)
MovieStar(<u>StarID</u>, Name, Gender)

Congratulations: You know select-project-join queries

- Very common subset to talk about
  - You saw (or will see) it in tutorial
- Can do many (but not all) useful things

SQL is *declarative*, not procedural
how do we know? Lets see what procedural would look like…

Conceptual Procedural Evaluation Strategy

1. Compute the cross-product of *relation-list*.
2. Discard resulting tuples if they fail *qualifications*.
3. Delete attributes that are not in *target-list*.
4. If DISTINCT is specified, eliminate duplicate rows.

SELECT	*target-list*
FROM	*relation-list*
WHERE	*qualification*

Example of Conceptual Procedural Evaluation

```
SELECT Name
FROM MovieStar M, StarsIn S
WHERE S.StarID = M.StarID AND
        MovieID = 276
```

StarID	Name	Gender	MovieID	StarID	Character
1273	Nathalie Portman	Female	272	1269	Leigh Anne Touhy
1273	Nathalie Portman	Female	273	1270	Mary
1273	Nathalie Portman	Female	274	1271	King George VI
1273	Nathalie Portman	Female	276	1273	Nina Sayers
…	…	…	…	…	…

New Students Example

- Class(<u>name</u>,meets_at,room,fid)
- Student(<u>snum</u>,sname,major,standing,age)
- Enrolled(<u>snum,cname</u>)
- Faculty(<u>fid</u>,fname,deptid)

Class Table

Name	Meets_at	Room	FID
Data Structures	MWF 10	R128	489456522
Database Systems	MWF 12:30-1:45	1320 DCL	142519864
Operating System Design	TuTh 12-1:20	20 AVW	489456522
Archaeology of the Incas	MWF 3-4:15	R128	248965255
Aviation Accident Investigation	TuTh 1-2:50	Q3	011564812
Air Quality Engineering	TuTh 10:30-11:45	R15	011564812
Introductory Latin	MWF 3-4:15	R12	248965255
American Political Parties	TuTh 2-3:15	20 AVW	619023588
Social Cognition	Tu 6:30-8:40	R15	159542516
Perception	MTuWTh 3	Q3	489221823
Multivariate Analysis	TuTh 2-3:15	R15	090873519
Patent Law	F 1-2:50	R128	090873519
Urban Economics	MWF 11	20 AVW	489221823
Organic Chemistry	TuTh 12:30-1:45	R12	489221823
Marketing Research	MW 10-11:15	1320 DCL	489221823
Seminar in American Art	M 4	R15	489221823
Orbital Mechanics	MWF 8 1320	DCL	011564812
Dairy Herd Management	TuTh 12:30-1:45	R128	356187925
Communication Networks	MW 9:30-10:45	20 AVW	141582651
Optical Electronics	TuTh 12:30-1:45	R15	254099823
Introduction to Math	TuTh 8-9:30	R128	489221823

Student Table

SNUM	SNAME	MAJOR	ST	AGE
51135593	Maria White	English	SR	21
60839453	Charles Harris	Architecture	SR	22
99354543	Susan Martin	Law	JR	20
112348546	Joseph Thompson	Computer Science	SO	19
115987938	Christopher Garcia	Computer Science	JR	20
132977562	Angela Martinez	History	SR	20
269734834	Thomas Robinson	Psychology	SO	18
280158572	Margaret Clark	Animal Science	FR	18
301221823	Juan Rodriguez	Psychology	JR	20
318548912	Dorthy Lewis	Finance	FR	18
320874981	Daniel Lee	Electrical Engineering	FR	17
322654189	Lisa Walker	Computer Science	SO	17
348121549	Paul Hall	Computer Science	JR	18
351565322	Nancy Allen	Accounting	JR	19
451519864	Mark Young	Finance	FR	18
455798411	Luis Hernandez	Electrical Engineering	FR	17
462156489	Donald King	Mechanical Engineering	SO	19
550156548	George Wright	Education	SR	21
552455318	Ana Lopez	Computer Engineering	SR	19
556784565	Kenneth Hill	Civil Engineering	SR	21
567354612	Karen Scott	Computer Engineering	FR	18
573284895	Steven Green	Kinesiology	SO	19
574489456	Betty Adams	Economics	JR	20
578875478	Edward Baker	Veterinary Medicine	SR	21

Enrolled Table

SNUM	CNAME
112348546	Database Systems
115987938	Database Systems
348121549	Database Systems
322654189	Database Systems
552455318	Database Systems
455798411	Operating System Design
552455318	Operating System Design
567354612	Operating System Design
112348546	Operating System Design
115987938	Operating System Design
322654189	Operating System Design
567354612	Data Structures
552455318	Communication Networks
455798411	Optical Electronics
455798411	Organic Chemistry
301221823	Perception
301221823	Social Cognition
301221823	American Political Parties
556784565	Air Quality Engineering
99354543	Patent Law
574489456	Urban Economics

Faculty Table

FID	FNAME	DEPTID
142519864	I. Teach	20
242518965	James Smith	68
141582651	Mary Johnson	20
011564812	John Williams	68
254099823	Patricia Jones	68
356187925	Robert Brown	12
489456522	Linda Davis	20
287321212	Michael Miller	12
248965255	Barbara Wilson	12
159542516	William Moore	33
090873519	Elizabeth Taylor	11
486512566	David Anderson	20
619023588	Jennifer Thomas	11
489221823	Richard Jackson	33
548977562	Ulysses Teach	20

Running Examples

Movie(MovieID, Title, Year)

StarsIn(MovieID, StarID, Character)

MovieStar(StarID, Name, Gender)

Student(snum,sname,major,standing,age)

Class(name,meets_at,room,fid)

Enrolled(snum,cname)

Faculty(fid,fname,deptid)

What kinds of queries can you answer so far?

Find the student ids of those who have taken a course named "Database Systems".

```
SELECT snum
FROM enrolled e
WHERE cname = 'Database Systems'
```

Do we need distinct? A. Yes. B. No

Student(snum,sname,major,standing,age)

Class(name,meets_at,room,fid)

Enrolled(snum,cname)

Faculty(fid,fname,deptid)

What kinds of queries can you answer so far?

Find the names of all classes taught by Elizabeth Taylor.

```
SELECT name
FROM Faculty f, class c
WHERE f.fid = c.fid and
      fname = 'Elizabeth Taylor'
```

Do we need distinct? A. Yes. B. No

Student(<u>snum</u>,sname,major,standing,age)

Class(<u>name</u>,meets_at,room,fid)

Enrolled(<u>snum,cname</u>)

Faculty(<u>fid</u>,fname,deptid)

What kinds of queries can you answer so far?

Find the departments that have at least one faculty member

```
SELECT DISTINCT deptid
FROM faculty
```

Do we need distinct? A. Yes. B. No

Student(<u>snum</u>,sname,major,standing,age)

Class(<u>name</u>,meets_at,room,fid)

Enrolled(<u>snum,cname</u>)

Faculty(<u>fid</u>,fname,deptid)

What kinds of queries can you answer so far?

Find the departments that have more than one faculty member (express not equal by "<>")

```
SELECT DISTINCT f1.deptid
FROM faculty f1, faculty f2
WHERE f1.fid <>f2.fid AND
      f1.deptid = f2.deptid
```

That is why renaming is important.

f1

fid	fname	Deptid
90873519	Elizabeth Taylor	11
619023588	Jennifer Thomas	11
…	…	…

f2

fid	fname	Deptid
90873519	Elizabeth Taylor	11
619023588	Jennifer Thomas	11
…	…	…

Do you need DISTINCT?

String Comparisons

What are the student ids of those who have taken a course with “System” in the name?

A string walks into a bar…

```
SELECT DISTINCT snum
FROM   enrolled
WHERE cname LIKE '%System%'
```

- LIKE is used for string matching:
  - '_' stands for any one character and
  - '%' stands for 0 or more arbitrary characters.
- SQL supports string operations such as
  - concatenation (using "||")
  - converting from upper to lower case (and vice versa)
  - finding string length, extracting substrings, etc.

A string walks into a bar…

```
SELECT DISTINCT snum
FROM   enrolled
WHERE cname LIKE '%System%'
```

Do we need DISTINCT?

A. Yes

B. No

Student(snum,sname,major,standing,age)

Class(name,meets_at,room,fid)

Enrolled(snum,cname)

Faculty(fid,fname,deptid)

Ordering of Tuples

List in alphabetic order the names of actors who were in a movie in 1939.

```
SELECT DISTINCT name
FROM Movie, StarsIn, MovieStar
WHERE Movie.MovieID = StarsIn.MovieID AND
      StarsIn.StarID = MovieStar.StarID AND
      year = 1939
ORDER BY name
```

Ordering of Tuples

Order is specified by:

- **desc** for descending order
- **asc** for ascending order (default)
- E.g. **order by** *Name* **desc**
- You can order within order: for example, … "ORDER BY Year, Name" would first order by Year, then Name within years

Clicker Question: Sorting

Relation R has schema R(a,b,c). In the result of the query

```
SELECT a, b, c
FROM R
ORDER BY c DESC, b ASC;
```

What condition must a tuple *t* satisfy so that *t* **necessarily precedes** the tuple (5,5,5)? Identify one such tuple from the list below.

A. (3,6,3)

B. (1,5,5)

C. (5,5,6)

D. All of the above

E. None of the above

Set Operations

- **union, intersect,** and **except** correspond to the relational algebra operations $\cup$, $\cap$, $-$.
- Each automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions:

  union all, intersect all and **except all**
- **A Union B** (OR) – combine the results from A and B
- **A Intersect B** (AND) – only keep results that appear in both A and B
- **A Except B** – only keep results found in A and not B

Set Operations

- Suppose a tuple occurs *m* times in *r* and *n* times in *s,* then, it occurs:
  - $m + n$ times in *r* **union all** *s*
  - $\min(m, n)$ times in *r* **intersect all** *s*
  - $\max(0, m - n)$ times in *r* **except all** *s*

Find IDs of MovieStars who've been in a movie in 1944 *or* 1974

- **UNION:** Can union any two *union-compatible* sets of tuples (i.e., the result of SQL queries).

```
SELECT  StarID
FROM  Movie M, StarsIn S
WHERE  M.MovieID=S.MovieID AND
( year = 1944 OR year = 1974)
```

- The two queries though quite similar return different results, why?
  - Use UNION ALL to get the same answer

```
SELECT  StarID
FROM     Movie M, StarsIn S
WHERE    M.MovieID = S.MovieID AND
year = 1944
UNION
SELECT  StarID
FROM     Movie M, StarsIn S
WHERE    M.MovieID = S.MovieID AND
year = 1974
```

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

Set Operations: Intersect

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

Example: Find IDs of stars who have been in a movie in 1944 *and* 1974.

```
SELECT  StarID
FROM    Movie M, StarsIn S
WHERE   M.MovieID = S.MovieID AND
year = 1944
INTERSECT
SELECT  StarID
FROM    Movie M, StarsIn S
WHERE   M.MovieID = S.MovieID AND
year = 1974
```

- **INTERSECT:** Can be used to compute the intersection of any two *union-compatible* sets of tuples.
- In SQL/92, but some systems don't support it.

Oracle does
MYSQL doesn't

Rewriting INTERSECT with Joins

Example: Find IDs of stars who have been in a movie in 1944 *and* 1974 without using **INTERSECT.**

```
SELECT  distinct S1.StarID
FROM    Movie M1, StarsIn S1,
        Movie M2, StarsIn S2
WHERE
        M1.MovieID = S1.MovieID AND M1.year = 1944 AND
        M2.MovieID = S2.MovieID AND M2.year = 1974 AND
        S2.StarID = S1.StarID
```

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

Clicker Question

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

Are these two queries equivalent?

A. Yes

B. No

```
SELECT  StarID
FROM    Movie M, StarsIn S
WHERE   M.MovieID =
S.MovieID AND year = 1944
INTERSECT
SELECT  StarID
FROM    Movie M, StarsIn S
WHERE   M.MovieID =
S.MovieID AND year = 1974
```

```
SELECT  StarID
FROM   Movie M, StarsIn S
WHERE   M.MovieID =
S.MovieID AND year =
1944 AND year = 1974
```

Set Operations: EXCEPT

Find the sids of all students who took Operating System Design but did not take Database Systems

```
SELECT snum
FROM enrolled e
WHERE cname = 'Operating System Design'
EXCEPT ← Oracle uses MINUS rather than EXCEPT
SELECT snum
FROM enrolled e
WHERE cname = 'Database Systems'
```

Student(snum,sname,major,standing,age)
Class(name,meets_at,room,fid)
Enrolled(snum,cname)
Faculty(fid,fname,deptid)

Can we do it in a different way?
(We'll come back to this)

But what about…

Select the IDs of all students who have not taken "Operating System Design"

- One way to do is to find all students that have taken a course.
- `MINUS` those who have taken "Operating System Design"

Student(snum,sname,major,standing,age)

Class(name,meets_at,room,fid)

Enrolled(snum,cname)

Faculty(fid,fname,deptid)

Motivating Example for Nested Queries

Find ids and names of female stars who have been in movie with ID 28:

```
SELECT M.StarID, name
FROM MovieStar M, StarsIn S
WHERE M.StarID = S.starID AND S.MovieID = 28
      AND gender = 'female';
```

Movie(<u>MovieID</u>, Title, Year)
StarsIn(<u>MovieID, StarID</u>, Character)
MovieStar(<u>StarID</u>, Name, Gender)

Motivating Example for Nested Queries

Find ids and names of female stars who have been in movie with ID 28:

```
SELECT M.StarID, name
FROM MovieStar M, StarsIn S
WHERE M.StarID = S.starID AND S.MovieID = 28
      AND gender = 'female';
```

Find ids and names of female stars who have not been in movie w/ ID 28 w/o using EXCEPT/MINUS:

- Would the following be correct?

```
SELECT M.StarID, name
FROM MovieStar M, StarsIn S
WHERE M.StarID = S.starID AND S.MovieID <> 28
      AND gender = 'female';
```

MovieStar

StarID	Name	Gender
1	Person A	Female
2	Person B	Male
3	Person C	Female

StarsIn

MovieID	StarID	Character
27	1	A tree
28	3	A coffee shop customer
28	2	A potato
29	3	A tired grad student

Movie(MovieID, Title, Year)
StarsIn(MovieID, StarID, Character)
MovieStar(StarID, Name, Gender)

```
SELECT M.StarID, name
FROM MovieStar M,
     StarsIn S
WHERE M.StarID =
      S.starID AND
      S.MovieID <> 28
      AND
      gender='female';
```

MovieStar m x StarsIn s

m.StarID	Name	Gender	MovieID	s.StarID	Character
1	Person A	Female	27	1	A tree
2	Person B	Male	27	1	A tree
3	Person C	Female	27	1	A tree
1	Person A	Female	28	3	A coffee shop customer
2	Person B	Male	28	3	A coffee shop customer
3	Person C	Female	28	3	A coffee shop customer
1	Person A	Female	28	2	A potato
2	Person B	Male	28	2	A potato
3	Person C	Female	28	2	A potato
1	Person A	Female	29	3	A tired grad student
2	Person B	Male	29	3	A tired grad student
3	Person C	Female	29	3	A tired grad student

MovieStar m x StarsIn s (processing m.starID = s.starID)

m.StarID	Name	Gender	MovieID	s.StarID	Character
1	Person A	Female	27	1	A tree
~~2~~	~~Person B~~	~~Male~~	~~27~~	~~1~~	~~A tree~~
~~3~~	~~Person C~~	~~Female~~	~~27~~	~~1~~	~~A tree~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~3~~	~~A coffee shop customer~~
3	Person C	Female	28	3	A coffee shop customer
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
2	Person B	Male	28	2	A potato
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~1~~	~~Person A~~	~~Female~~	~~29~~	~~3~~	~~A tired grad student~~
~~2~~	~~Person B~~	~~Male~~	~~29~~	~~3~~	~~A tired grad student~~
3	Person C	Female	29	3	A tired grad student

MovieStar m x StarsIn s (processing gender = “female”)

m.StarID	Name	Gender	MovieID	s.StarID	Character
1	Person A	Female	27	1	A tree
~~2~~	~~Person B~~	~~Male~~	~~27~~	~~1~~	~~A tree~~
~~3~~	~~Person C~~	~~Female~~	~~27~~	~~1~~	~~A tree~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~3~~	~~A coffee shop customer~~
3	Person C	Female	28	3	A coffee shop customer
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~2~~	~~A potato~~
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~1~~	~~Person A~~	~~Female~~	~~29~~	~~3~~	~~A tired grad student~~
~~2~~	~~Person B~~	~~Male~~	~~29~~	~~3~~	~~A tired grad student~~
3	Person C	Female	29	3	A tired grad student

MovieStar m x StarsIn s (processing movieID <> 28)

m.StarID	Name	Gender	MovieID	s.StarID	Character
1	Person A	Female	27	1	A tree
~~2~~	~~Person B~~	~~Male~~	~~27~~	~~1~~	~~A tree~~
~~3~~	~~Person C~~	~~Female~~	~~27~~	~~1~~	~~A tree~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~2~~	~~A potato~~
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~1~~	~~Person A~~	~~Female~~	~~29~~	~~3~~	~~A tired grad student~~
~~2~~	~~Person B~~	~~Male~~	~~29~~	~~3~~	~~A tired grad student~~
3	Person C	Female	29	3	A tired grad student

Processing the SELECT

m.StarID	Name	Gender	MovieID	s.StarID	Character
1	Person A	Female	27	1	A tree
~~2~~	~~Person B~~	~~Male~~	~~27~~	~~1~~	~~A tree~~
~~3~~	~~Person C~~	~~Female~~	~~27~~	~~1~~	~~A tree~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~3~~	~~A coffee shop customer~~
~~1~~	~~Person A~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~2~~	~~Person B~~	~~Male~~	~~28~~	~~2~~	~~A potato~~
~~3~~	~~Person C~~	~~Female~~	~~28~~	~~2~~	~~A potato~~
~~1~~	~~Person A~~	~~Female~~	~~29~~	~~3~~	~~A tired grad student~~
~~2~~	~~Person B~~	~~Male~~	~~29~~	~~3~~	~~A tired grad student~~
3	Person C	Female	29	3	A tired grad student

Nested Queries

- A very powerful feature of SQL:

```
SELECT  A1,A2,…,An
FROM    R1,R2, …,Rm
WHERE   condition
```

- A nested query is a query that has another query embedded with it.
  - A **SELECT, FROM, WHERE, or HAVING** clause can itself contain an SQL query!
  - Being part of the WHERE clause is the most common

Nested Queries (IN/Not IN)

Find ids and names of stars who have been in movie with ID 28:

Nested Queries (IN/Not IN)

Find ids and names of stars who have been in movie with ID 28:

```
SELECT M.StarID, M.Name
FROM MovieStar M
WHERE M.Gender =  'female' AND
      M.StarID IN  (SELECT  S.StarID
                    FROM  StarsIn S
                    WHERE  MovieID = 28)
```

There's also NOT IN

- To find stars who have *not* been in movie 28, use **NOT IN**.
- To understand nested query semantics, think of a *nested loops* evaluation:
  - *For each MovieStar tuple, check the qualification by computing the subquery.*

Nested Queries (IN/Not IN)

Find ids and names of stars who have been in movie with ID 28:

```
SELECT M.StarID, M.Name
FROM MovieStar M
WHERE M.Gender =  'female' AND
      M.StarID IN  (SELECT  S.StarID
                    FROM  StarsIn S
                    WHERE  MovieID = 28)
```

- In this example in inner query does not depend on the outer query so it could be computed just once.
- Think of this as a function that has no parameters

```
SELECT S.StarID
FROM StarsIn S
WHERE MovieID=28
```

StarID
1026
1027

```
SELECT M.StarID, M.Name
FROM MovieStar M
WHERE M.Gender = 'female' AND
      M.StarID IN (1026,1027)
```

Rewriting EXCEPT Queries Using In

Using nested queries, find the sids of all students who took Operating System Design but did not take Database Systems.

Rewriting INTERSECT Queries Using IN

Find IDs of stars who have been in movies in 1944 and 1974.

```
SELECT S.StarID
FROM Movie M, StarsIn S
WHERE M.MovieID = S.MovieID AND
      M.year = 1944 AND
      S.StarID IN
         (SELECT  S2.StarID
          FROM Movie M2, StarsIn S2
          WHERE M2.MovieID = S2.MovieID
                AND M2.year = 1974)
```

The subquery finds stars who have been in movies in 1974.

Nested Queries with Correlation
Same idea, subtle difference

Find names of stars who have been in movie w/ ID 28:

```
SELECT M.Name
FROM MovieStar M
WHERE EXISTS  (SELECT *
               FROM  StarsIn S
               WHERE  MovieID=28 AND
                      S.StarID = M.StarID)
```

- **EXISTS**: *returns true if the set is not empty*.
- **UNIQUE**: *returns true if there are no duplicates*.
- Illustrates why, in general, subquery must be re-computed for each StarsIn tuple.

SQL EXISTS Condition

- The `SQL EXISTS` condition is used in combination with a subquery and is considered to be met if the subquery returns at least one row.

- It can be used in a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.

- We can also use `NOT EXISTS`

SQL EXISTS Condition

Using the `EXISTS`/`NOT EXISTS` operations and correlated queries, find the name and age of the oldest student(s).

```
SELECT sname, age
FROM student s2
WHERE NOT EXISTS(SELECT *
                 FROM student s1
                 WHERE s1.age > s2.age)
```

Does there exist a tuple in s1 such that the age of the s1 tuple is greater than the age of the tuple in s2?

SQL EXISTS Condition

```
SELECT sname, age
FROM student s2
WHERE NOT EXISTS(SELECT *
                 FROM student s1
                 WHERE s1.age > s2.age)
```

Student s2

snum	Name	Age
1	Bluey	6
2	Chilli	9
3	Bingo	4

More on Set-Comparison Operators

- We've already seen **IN and EXISTS**. Can also use **NOT IN**, **NOT EXISTS**.
- Also available: **op ANY**, **op ALL**
  where op is one of: **>, <, =, <=, >=, <>**

More on Set-Comparison Operators

- We've already seen **IN and EXISTS**. Can also use **NOT IN, NOT EXISTS**.
- Also available: **op ANY, op ALL** where op is one of: **>, <, =, <=, >=, <>**

Find movies made after "Fargo".

```
SELECT *
FROM Movie
WHERE year > ANY (SELECT  year
                  FROM   Movie
                  WHERE  Title ='Fargo')
```

Just returning one column

If we have multiple movies named Fargo then we can use ALL instead of ANY

Clicker Question

```
SELECT Team, Day
FROM Scores S1
WHERE Runs <= ALL
  (SELECT Runs
   FROM Scores S2
   WHERE
     S1.Day = S2.Day)
```

Which of the following is in the result:

A. (Carp, Sun)

B. (Bay Stars, Sun)

C. (Swallows, Mon)

D. All of the above

E. None of the above

Scores:

Team	Day	Opponent	Runs
Dragons	Sun	Swallows	4
Tigers	Sun	Bay Stars	9
Carp	Sun	Giants	2
Swallows	Sun	Dragons	7
Bay Stars	Sun	Tigers	2
Giants	Sun	Carp	4
Dragons	Mon	Carp	6
Tigers	Mon	Bay Stars	5
Carp	Mon	Dragons	3
Swallows	Mon	Giants	0
Bay Stars	Mon	Tigers	7
Giants	Mon	Swallows	5

Example

You can rewrite queries that use any or all with queries that use exist or not exist.

Using the any or all operations, find the name and age of the oldest student(s).

```
SELECT sname, age
FROM student s2
WHERE NOT EXISTS (SELECT *
                  FROM student s1
                  WHERE s1.age >s2.age)
```

```
SELECT sname, age
FROM student s2
WHERE s2.age >= ALL (SELECT age
                     FROM student s1)
```

Clicker Question

Consider the following SQL query

```
SELECT DISTINCT s1.sname, s1.age
FROM student s1, student s2
WHERE s1.age > s2.age
```

This query returns

A. The name and age of one of the oldest student(s)
B. The name and age of all of the oldest student(s)
C. The name and age of all of the youngest student(s)
D. The name and age of all students that are older than the youngest student(s)
E. None of the above

Division

- Useful for expressing queries that include a notion of "**for all**" or "**for every**"

- E.g., *Find movie stars who were in **all** movies*.

Examples for Division

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

B1

pno
p2

B2

pno
p2
p4

B3

pno
p1
p2
p4

A/B1

sno
s1
s2
s3
s4

A/B2

sno
s1
s4

A/B3

sno
s1

Division in SQ

Find students who've taken all classes.

```
SELECT sname
FROM   Student S
WHERE NOT EXISTS
              ((SELECT  C.name
                 FROM  Class C)
                 EXCEPT
                  (SELECT  E.cname
                   FROM  Enrolled E
                   WHERE E.snum=S.snum))
```

(method 1)

The hard way (without EXCEPT):

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
      SELECT  C.name
      FROM  Class C
      WHERE  NOT EXISTS (SELECT  E.snum
                         FROM  Enrolled E
                         WHERE  C.name= E.cname AND
                                E.snum=S.snum))
```

(method 2)

Division in SQL (method 1- use EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
    (SELECT C.name
     FROM  Class C)
     EXCEPT
     (SELECT  E.cname
      FROM  Enrolled E
      WHERE E.snum=S.snum))
```

All classes

Division in SQL (method 1- use EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
    (SELECT C.name
     FROM  Class C)
     EXCEPT
     (SELECT  E.cname
      FROM  Enrolled E
      WHERE E.snum=S.snum))
```

All classes taken by S

Division in SQL (method 1- use EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
    (SELECT C.name
     FROM  Class C)
     EXCEPT
     (SELECT  E.cname
      FROM  Enrolled E
      WHERE E.snum=S.snum))
```

All classes that have not been taken by S

Division in SQL (method 1- use EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
    (SELECT C.name
     FROM  Class C)
     EXCEPT
      (SELECT  E.cname
       FROM  Enrolled E
       WHERE E.snum=S.snum))
```

Only true if there is no class that has not been taken by S (i.e., S must have taken all the classes).

Division in SQL (method 2- without using EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
   SELECT  C.name
   FROM  Class C
   WHERE  NOT EXISTS (SELECT  E.snum
                      FROM  Enrolled E
                      WHERE C.name=E.cname AND
                            E.snum=S.snum))
```

Returns a result if student S is enrolled in class C

Division in SQL (method 2- without using EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
   SELECT  C.name
   FROM  Class C
   WHERE  NOT EXISTS (SELECT  E.snum
                      FROM  Enrolled E
                      WHERE C.name=E.cname AND
                            E.snum=S.snum))
```

Only true if student S has never been enrolled in class C.

Division in SQL (method 2- without using EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
   SELECT  C.name
   FROM  Class C
   WHERE  NOT EXISTS (SELECT  E.snum
                      FROM  Enrolled E
                      WHERE C.name=E.cname AND
                            E.snum=S.snum))
```

Find the classes that student S has not enrolled in.

Division in SQL (method 2- without using EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
   SELECT  C.name
   FROM  Class C
   WHERE  NOT EXISTS (SELECT  E.snum
                      FROM  Enrolled E
                      WHERE C.name=E.cname AND
                            E.snum=S.snum))
```

Only true if there is no class that student S has never been enrolled in (i.e., student S has been enrolled in all the classes).

Division in SQL (method 2- without using EXCEPT)

Find students who've taken all classes.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
   SELECT  C.name
   FROM  Class C
   WHERE  NOT EXISTS (SELECT  E.snum
                      FROM  Enrolled E
                      WHERE C.name=E.cname AND
                            E.snum=S.snum))
```

Select Student S such that there is no class C which is not taken by S.

```
SELECT sname
FROM Student S
WHERE NOT EXISTS (
    SELECT  C.name
    FROM  Class C
    WHERE  NOT EXISTS (SELECT  E.snum
                       FROM  Enrolled E
                       WHERE C.name=E.cname AND
                             E.snum=S.snum))
```

Student S

snum	sname
1	Jessica
2	Rachel
3	Kemi

Class C

name
CPSC 100
CPSC 304
CPSC 368

Enrolled E

cname	snum
CPSC 100	Jessica
CPSC 100	Rachel
CPSC 100	Kemi
CPSC 304	Rachel
CPSC 368	Jessica
CPSC 304	Jessica

Aggregate Operators

- These functions operate on the multiset of values of a column of a relation, and return a value

  AVG: average value
  MIN: minimum value
  MAX: maximum value
  SUM: sum of values
  COUNT: number of values

```
SELECT count(s.snum)
FROM enrolled e, Student S
WHERE e.snum = s.snum
```

- The following versions eliminate duplicates before applying the operation to attribute A:

  COUNT (DISTINCT A)
  SUM (DISTINCT A)
  AVG (DISTINCT A)

```
SELECT count(distinct s.snum)
FROM enrolled e, Student S
WHERE e.snum = s.snum
```

Aggregate Operators: Examples

AVG: average value
MIN: minimum value
MAX: maximum value
SUM: sum of values
COUNT: number of values

students

```
SELECT   COUNT(*)
FROM     Student
```

Find name and age of the oldest student(s).

```
SELECT sname, age
FROM Student S
WHERE S.age = (SELECT  MAX(S2.age)
               FROM  Student S2)
```

Finding average age of SR students.

```
SELECT AVG (age)
FROM Student
WHERE standing='SR'
```

Aggregation Examples

AVG: average value
MIN: minimum value
MAX: maximum value
SUM: sum of values
COUNT: number of values

Find the minimum student age.

```
SELECT MIN(age)
FROM student;
```

How many students have taken a class with "Database" in the title.

GROUP BY and HAVING

- Divide tuples into groups and apply aggregate operations to each group.
- Example: *Find the age of the youngest student for each major.*

For i = 'Computer Science',
'Civil Engineering'…

```
SELECT  MIN (age)
FROM     Student
WHERE  major = i
```

Problem:

We don't know how many majors exist, not to mention this is not good practice

Grouping Examples

Find the age of the youngest student who is at least 19, for each major.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…

No Animal Science

Major	Age
Computer Science	19
Accounting	19
Civil Engineering	21
…	…

Grouping Examples with Having

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…

Grouping Examples with Having

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

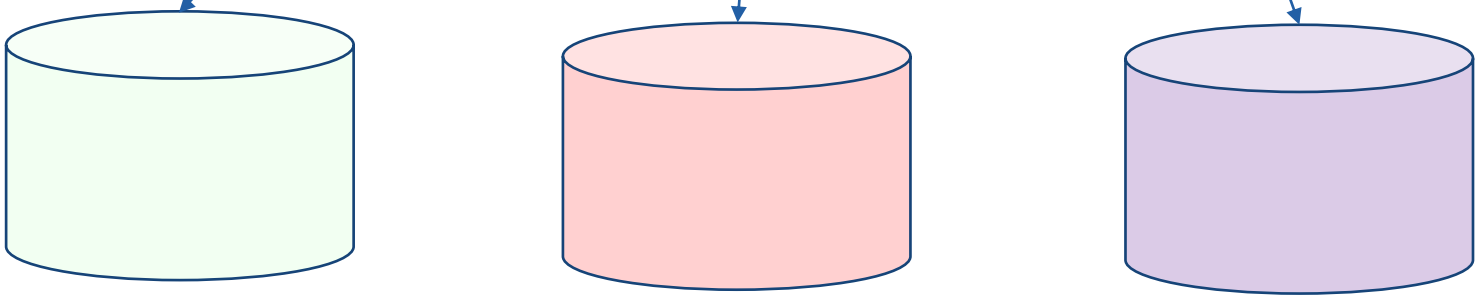
Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…

Grouping Examples with Having

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…

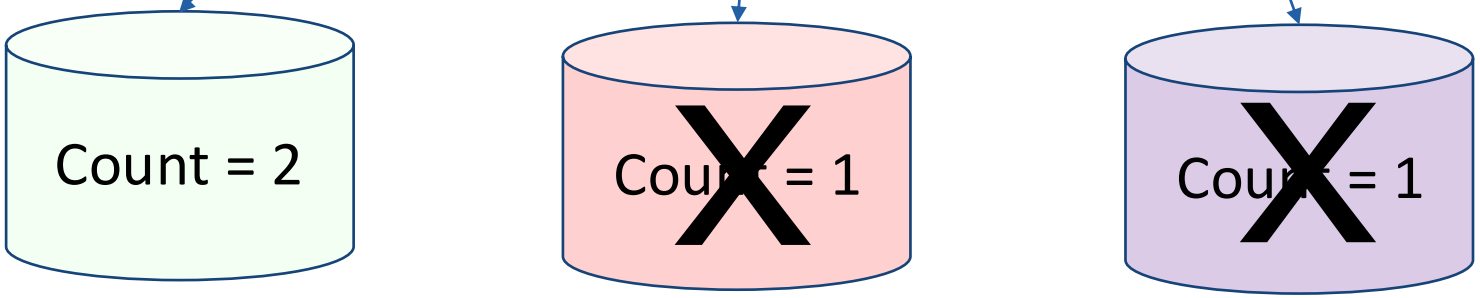


Grouping Examples with Having

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…



Grouping Examples with Having

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…

Major	Age
Computer Science	19

And there are rules

Find the age of the youngest student who is at least 19, for each major with at least 2 such students

```
SELECT      major, MIN(age)
FROM        Student
WHERE       age >= 19
GROUP BY    major
HAVING      COUNT(*) > 1
```

- Would it make sense if I select age instead of MIN(age)?
- *Would it make sense if I select snum to be returned?*
- *Would it make sense if I select major to be returned?*

And there are rules

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

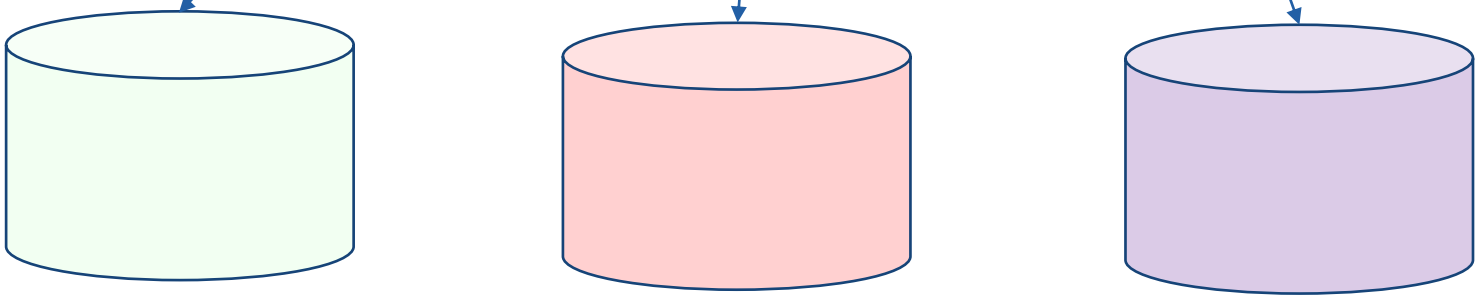
- Would it make sense if I select `age` instead of `MIN(age)`?
- Would it make sense if I select `snum` to be returned?
- Would it make sense if I select `major` to be returned?

And there are rules

Find the age of the youngest student who is at least 19, for each major with at least 2 <u>such</u> students.

```
SELECT major, MIN(age)
FROM Student
WHERE age >= 19
GROUP BY major
HAVING COUNT(*) > 1
```

Snum	Major	Age
115987938	Computer Science	20
112348546	Computer Science	19
280158572	Animal Science	18
351565322	Accounting	19
556784565	Civil Engineering	21
…	…	…



GROUP BY and HAVING (cont)

```
SELECT [DISTINCT]  target-list
FROM relation-list
WHERE qualification
GROUP BY grouping-list
HAVING group-qualification
ORDER BY target-list
```

- The *target-list* contains
  (i) attribute names
  (ii) terms with aggregate operations (e.g., `MIN(S.age)`).
- Attributes in (i) must also be in *grouping-list*.
  - each answer tuple corresponds to a *group*,
  - *group* = a set of tuples with same value for all attributes in *grouping-list*
  - selected attributes must have a single value per group.
- Attributes in *group-qualification* are either in *grouping-list* or are arguments to an aggregate operator.

Conceptual Evaluation of a Query

1. compute the cross-product of *relation-list* from
2. keep only tuples that satisfy *qualification* where
3. partition the remaining tuples into groups by the value of attributes in *grouping-list* group by
4. keep only the groups that satisfy *group-qualification* (expressions in *group-qualification* must have a *single value per group*!) having
5. delete fields that are not in *target-list* select
6. generate one answer tuple per qualifying group.

GROUP BY and HAVING

Student(snum,sname,major,standing,age)
Class(cname,meets_at,room,fid)
Enrolled(snum,cname)
Faculty(fid,fname,deptid)

Example 1: For each class, find the age of the youngest student who has enrolled in this class.

```
SELECT cname, MIN(age)
FROM Student S, Enrolled E
WHERE S.snum= E.snum
GROUP BY cname
```

Example 2: For each course with more than 1 enrollment, find the age of the youngest student who has taken this class.

```
SELECT cname,  MIN(age)
FROM Student S, Enrolled E
WHERE S.snum = E.snum
GROUP BY cname
HAVING COUNT(*) > 1
```

← per group qualification!

Clicker Question: Grouping

clickergrouping.sql

Compute the result of the query:

```
SELECT a1.x, a2.y, COUNT(*)
FROM Arc a1, Arc a2
WHERE a1.y = a2.x
GROUP BY a1.x, a2.y
```

Which of the following is in the result?

A. (1,3,2)

B. (4,2,6)

C. (4,3,1)

D. All of the above

E. None of the above

Arc

x	y
1	2
1	2
2	3
3	4
3	4
4	1
4	1
4	1
4	2

Groupies of your very own

Find the average age for each standing (e.g., Freshman).

```
SELECT standing, avg(age)
FROM Student
GROUP BY standing
```

Student(snum,sname,major,standing,age)

Class(name,meets_at,room,fid)

Enrolled(snum,cname)

Faculty(fid,fname,deptid)

Groupies of your very own

Find the deptID and # of faculty members for each department having a department id > 20

Option 1:

```
SELECT count(*), deptid
FROM faculty
WHERE deptid > 20
GROUP BY deptid
```

Option 2:

```
SELECT count(*), deptid
FROM faculty
GROUP BY deptid
HAVING deptid > 20
```

Which one works?

A: Just 1
B: Just 2
C: Both
D: Neither

Grouping Examples (cont')

For each standing, find the number of students who took a class with "System" in the title

```
SELECT s.standing, COUNT(DISTINCT s.snum) AS scount
FROM Student S, enrolled E
WHERE S.snum = E.snum and E.cname like '%System%'
GROUP BY s.standing
```

What if we do the following:
a) remove *E.cname like '%System%'* from the WHERE clause, and then

b) add a HAVING clause with the dropped condition?

```
SELECT   s.standing,  COUNT(DISTINCT s.snum) AS scount
FROM       Student S, enrolled E
WHERE    S.snum = E.snum
GROUP BY   s.standing
HAVING   E.cname like '%System%'
```

E.Cname not in groupby Error!

Clicker Question: Having

Suppose we have a relation with schema R(A, B, C, D, E). If we issue a query of the form:

```
SELECT ...
FROM R
WHERE ...
GROUP BY B, E
HAVING ???
```

Identify, in the list below, the term that CANNOT appear in the HAVING clause (where the ??? is).

A. A (unaggregated)

B. B (unaggregated)

C. Count(B)

D. All can appear

E. None can appear

Grouping Examples (cont')

Find the age of the youngest student with age > 18, for each major with at least 2 students(of age > 18).

```
SELECT S.major, MIN(S.age)
FROM   Student S
WHERE  S.Age > 18
GROUP BY S.major
HAVING COUNT(*) > 1
```

Student(snum,sname,major,standing,age)
Class(name,meets_at,room,fid)
Enrolled(snum,cname)
Faculty(fid,fname,deptid)

Grouping Examples (cont')

Find the age of the youngest student with age > 18, for each major for which the average age of the students who are >18 is higher than the average age of all students across all majors.

Grouping Examples (cont')

Find the age of the youngest student with age > 18, for each major for which the average age of the students who are >18 is higher than the average age of all students across all majors.

```
SELECT S.major, MIN(S.age), avg(S.age)
FROM  Student S, Student S2
WHERE  S.age > 18 AND S.snum = S2.snum
GROUP BY  S.major
HAVING avg(S.age) > avg(S2.age)
```

Does this query perform the stated task?

A. Yes B. No

Grouping Examples (cont')

Student table (some attributes omitted)

snum	major	age
1	CS	18
2	Music	20
3	Music	19
4	English	17
5	Business	21

Joining two instances of student and removing S.age <= 18

S.snum	S.major	S.age	S2.snum	S2.major	S2.age
~~1~~	~~CS~~	~~18~~	~~1~~	~~CS~~	~~18~~
2	Music	20	2	Music	20
3	Music	19	3	Music	19
~~4~~	~~English~~	~~17~~	~~4~~	~~English~~	~~17~~
5	Business	21	5	Business	21

Grouping Examples (cont')

Grouping by major (each group shown separately)

S.snum	S.major	S.age	S2.snum	S2.major	S2.age
2	Music	20	2	Music	20
3	Music	19	3	Music	19

S.snum	S.major	S.age	S2.snum	S2.major	S2.age
5	Business	21	5	Business	21

Taking the average age of S.age would be the same as taking the average age of S2.age.

Grouping Examples (cont')

Find the age of the youngest student with age > 18, for each major with at least 2 students(of any age)

```
SELECT S.major, MIN(S.age)
FROM Student S
WHERE S.age > 18
GROUP BY S.major
HAVING 1 < (SELECT  COUNT(*)
            FROM  Student S2
            WHERE  S.major=S2.major)
```

- Subqueries in the HAVING clause can be correlated with fields from the outer query.

Grouping Examples (cont')

Find those majors for which their average age is the minimum over all majors

```
SELECT major, avg(age)
FROM  student S
GROUP BY major
HAVING min(avg(age))
```

- WRONG, cannot use nested aggregation
  - One solution would be to use subquery in the FROM Clause

```
SELECT  Temp.major, Temp.average
FROM (SELECT  S.major, AVG(S.age) as average
      FROM  Student S
      GROUP BY S.major) AS Temp
WHERE Temp.average in (SELECT  MIN(Temp.average)
                       FROM  Temp)
```

A bit ugly

Grouping Examples (cont')

Find those majors for which their average age is the minimum over all majors

```
SELECT major, avg(age)
FROM  student S
GROUP BY major
HAVING min(avg(age))
```

- WRONG, cannot use nested aggregation
  - One solution would be to use subquery in the FROM Clause

```
SELECT major, avg(age)
FROM  student S
GROUP BY major
HAVING AVG(age) <= ALL (SELECT  AVG(S.age)
                        FROM  Student S
                        GROUP BY S.major)
```

Easiest method would be to use Views.

What are views

- Relations that are defined with a `CREATE TABLE` statement exist in the physical layer
  - Do not change unless explicitly told so

- Virtual views do not physically exist, they are defined by expression over the tables.
  - Can be queries (most of the time) as if they were tables.

Why use views?

- Hide some data from users
- Make some queries easier
- Modularity of database
- When not specified exactly based on tables.

Example: UBC has one table for students. Should the CS Department be able to update CS students info? Yes.

Biology students? NO

Create a view for CS to only be able to update CS students

Defining and using Views

- `Create View <view name><attributes in view> As <view definition>`
  - View definition is defined in SQL
  - From now on we can use the view almost as if it is just a normal table
- View V (R_1,…R_n)
- query Q involving V
  - Conceptually
    - V (R_1,…R_n) is used to evaluate Q
  - In reality
    - The evaluation is performed over R_1,…R_n

Defining and using Views

Suppose you had tables

Course(Course#,title,dept)

Enrolled(Course#,sid,mark)

```
CREATE  VIEW  CourseWithFails(dept,
course#, mark) AS
    SELECT C.dept, C.course#, mark
    FROM Course C, Enrolled E
    WHERE C.course# = E.course# AND
    mark<50
```

This view gives the dept, course#, and marks for those courses where someone failed.

Views and Security

- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).
- Given CourseWithFails, but not Course or Enrolled, we can find the course in which some students failed, but we can't find the students who failed.

```
Course(Course#,title,dept)
Enrolled(Course#,sid,mark)
VIEW  CourseWithFails(dept, course#, mark)
```

View Updates

- View updates must occur at the base tables.
  - Ambiguous
  - Difficult

CourseWithFails(dept, course#, mark)

Course(Course#,title,dept)
Enrolled(Course#,sid,mark)

- DBMS's restrict view updates only to some simple views on single tables (called updatable views)

View Deletes

- `DROP VIEW <view name>`
  - Dropping a view does not affect any tuples of the in the underlying relation.

- How to handle DROP TABLE if there's a view on the table?
- DROP TABLE command has options to prevent a table from being dropped if views are defined on it:
  - DROP TABLE Student RESTRICT
    - drops the table, unless there is a view on it
  - DROP TABLE Student CASCADE
    - drops the table, and recursively drops any view referencing it

The Beauty of Views

Find those majors for which their average age is the minimum over all majors.

With views:

```
CREATE VIEW Temp(major, average) as
   SELECT S.major, AVG(S.age) AS average
   FROM Student S
   GROUP BY  S.major;

SELECT major, average
FROM Temp
WHERE average = (SELECT MIN(average) FROM Temp)
```

Without views:

A bit ugly

```
SELECT Temp.major, Temp.average
FROM (SELECT  S.major, AVG(S.age) as average
      FROM  Student S
      GROUP BY S.major) AS Temp
WHERE Temp.average in (SELECT  MIN(Temp.average)
                       FROM  Temp)
```

Clicker Question: Views

Suppose relation `R(a,b,c)`:

Define the view *V* by:

```
CREATE VIEW V AS
    SELECT a+b AS d, c
    FROM R;
```

What is the result of the query:

```
SELECT d, SUM(c)
FROM V
GROUP BY d
HAVING COUNT(*) <> 1;
```

R

a	b	c
1	1	3
1	2	3
2	1	4
2	3	5
2	4	1
3	2	4
3	3	6

V

d	c
2	3
3	3
3	4
5	5
6	1
5	4
6	6

d	sum(c)
3	7
5	9
6	7

Identify, from the list below, a tuple in the result of the query:

A. (2,3)

B. (3,12)

C. (5,9)

D. All are correct

E. None are correct

Null Values

- Tuples may have a null value, denoted by *null*, for some of their attributes
- Value *null* signifies an unknown value or that a value does not exist.
- The predicate **IS NULL** (**IS NOT NULL**) can be used to check for null values.
  - E.g., *Find all student names whose age is not known.*

```
SELECT name
FROM Student
WHERE age IS NULL
```

- The result of any arithmetic expression involving *null* is *null*
  - E.g., 5 + *null* returns *null.*

Null Values and Three Valued Logic

- null requires a 3-valued logic using the truth value *unknown*:
  - OR: (*unknown* **or** *true*) = *true*, (*unknown* **or** *false*) = *unknown*
    (*unknown* **or** *unknown*) = *unknown*
  - AND: *(true* **and** *unknown) = unknown, (false* **and** *unknown) = false,*
    (unknown **and** *unknown) = unknown*
  - NOT*: (***not** *unknown) = unknown*
  - "*P* **is unknown"** evaluates to true if predicate *P* evaluates to *unknown*
- Any comparison with *null* returns *unknown*
  - *E.g. 5 < null or null <> null or null = null*
- Result of **where** clause predicate is treated as *false* if it evaluates to *unknown*
- All aggregate operations except **count(*)** ignore tuples with null values on the aggregated attributes.

```
SELECT count(*)        SELECT count(fid)
FROM class             FROM class
```

Clicker Question: Null

```
SELECT COUNT(*),
       COUNT(Runs)
FROM Scores
WHERE Team = 'Carp'
```

Which of the following is in the result:

A. (1,0)

B. (2,0)

C. (1,NULL)

D. All of the above

E. None of the above

Scores

Team	Day	Opponent	Runs
Dragons	Sun	Swallows	4
Tigers	Sun	Bay Stars	9
Carp	Sun	NULL	NULL
Swallows	Sun	Dragons	7
Bay Stars	Sun	Tigers	2
Giants	Sun	NULL	NULL
Dragons	Mon	Carp	NULL
Tigers	Mon	NULL	NULL
Carp	Mon	Dragons	NULL
Swallows	Mon	Giants	0
Bay Stars	Mon	NULL	NULL
Giants	Mon	Swallows	5

Natural Join

- The SQL NATURAL JOIN is a type of EQUI JOIN and is structured in such a way that, columns with same name of associate tables will appear once only.
- Natural Join : Guidelines
  - The associated tables have one or more pairs of identically named columns.
  - The columns must be the same data type.
  - Don't use ON clause in a natural join.

```
SELECT *
FROM student s natural join enrolled e
```

- Natural join of tables with no pairs of identically named columns will return the cross product of the two tables.

```
SELECT *
FROM student s natural join class c
```

More fun with joins

- What happens if I execute query:

```
SELECT *
FROM student s, enrolled e
WHERE s.snum = e.snum
```

- To get *all* students, you need an *outer join*
- There are several special joins declared in the *FROM* clause:
  - Inner join – default: only include matches
  - Left outer join – include all tuples from left hand relation
  - Right outer join – include all tuples from right hand relation
  - Full outer join – include all tuples from both relations

Example:

```
SELECT  *
FROM  Student S  NATURAL LEFT OUTER JOIN
Enrolled E
```

More fun with joins examples

R

A	B
1	2
3	3

S

B	C
2	4
4	6

Natural Inner Join

A	B	C
1	2	4

Natural Left outer Join

A	B	C
1	2	4
3	3	Null

Natural Right outer Join

A	B	C
1	2	4
Null	4	6

Natural outer Join

A	B	C
1	2	4
3	3	Null
Null	4	6

Outer join (without the Natural) will use the key word ON for specifying the condition of the join.

Outer join not implemented in MYSQL
Outer join is implemented in Oracle

Clicker Question: Outer

Compute:

```
SELECT R.A, R.B, S.B, S.C, S.D
FROM R FULL OUTER JOIN S
   ON (R.A > S.B AND R.B = S.C)
```

R(A,B)

A	B
1	2
3	4
5	6

S(B,C,D)

B	C	D
2	4	6
4	6	8
4	7	9

Which of the following tuples of R or S is dangling (and therefore needs to be padded in the outer join)?

A. (1,2) of R

B. (3,4) of R

C. (2,4,6) of S

D. All of the above

E. None of the above

A	B	B	C	D
3	4	2	4	6
5	6	4	6	8
1	2	NULL	NULL	NULL
NULL	NULL	4	7	9

Database Manipulation Insertion redux

Can insert a single tuple using:

```
INSERT  INTO  Student VALUES
   (53688, 'Smith', '222 W.15th ave',
   333-4444, MATH)
```

or

```
INSERT  INTO
  Student (sid, name, address, phone,
  major) VALUES  (53688, 'Smith', '222
  W.15th ave', 333-4444, MATH)
```

Add a tuple to student with null address and phone:

```
INSERT  INTO  Student (sid, name,
address, phone, major) VALUES  (33388,
'Chan', null, null, CPSC)
```

Database Manipulation Insertion redux (cont)

- Can add values selected from another table

- Enroll student 51135593 into every class taught by faculty 90873519

```
INSERT  INTO  Enrolled
     SELECT  51135593, name
     FROM Class
     WHERE fid = 90873519
```

The SELECT-FROM-WHERE statement is fully evaluated before any of its results are inserted or deleted.

Database Manipulation Deletion

- Note that only whole tuples are deleted.
- Can delete all tuples satisfying some condition (e.g., name = Smith):

```
DELETE FROM Student
WHERE name = 'Smith'
```

- The WHERE clause can contain nested queries

Database Manipulation Updates

- Increase the age of all students by 2 (should not be more than 100)
- Need to write two updates:

```
UPDATE Student
SET age = 100
WHERE age >= 98

UPDATE Student
SET age = age + 2
WHERE age < 98
```

- Is the order important?

A: Yes

B: No

Integrity Constraints (Review)

- An IC describes conditions that every *legal instance* of a relation must satisfy.
  - Inserts/deletes/updates that violate IC's are disallowed.
  - Can ensure application semantics (e.g., *sid* is a key), or prevent inconsistencies (e.g., *sname* has to be a string, *age* must be < 200)

- *Types of IC's*:
  - domain constraints,
  - primary key constraints,
  - foreign key constraints,
  - general constraints

General Constraints: Check

We can specify constraints over a single table using table constraints, which have the form:

```
CHECK conditional-expression
```

```
CREATE TABLE   Student
     (snum  INTEGER,
      sname  CHAR(32),
      major  CHAR(32),
      standing CHAR(2),
      age  REAL,
      PRIMARY KEY  (snum),
      CHECK  (age >= 10 AND age < 100);
```

Check constraints are checked when tuples are inserted or modified

General Constraints: Check

- Constraints can be named
- Can use subqueries to express constraint
- Table constraints are associated with a single table, although the conditional expression in the check clause can refer to other tables

No one can be enrolled in a class, which is held in R15

```
CREATE TABLE  Enrolled
    (snum INTEGER,
     cname   CHAR(32),
     PRIMARY KEY  (snum, cname),
     CONSTRAINT  noR15
        CHECK  (`R15' <>
              (SELECT c.room
               FROM     class c
               WHERE  c.name=cname)));
```

Constraints Over Multiple Relations

- Cannot be defined in one table.
- Are defined as ASSERTIONs which are not associated with any table

- Example: *Every MovieStar needs to star in at least one Movie*

```
CREATE ASSERTION  totalEmployment
CHECK
(NOT EXISTS ((SELECT StarID FROM MovieStar)
             EXCEPT
             (SELECT StarID FROM StarsIn)));
```

Constraints Over Multiple Relations

Example: Write an assertion to enforce every student to be registered in at least one course.

Student(snum,sname,major,standing,age)
Class(name,meets_at,room,fid)
Enrolled(snum,cname)
Faculty(fid,fname,deptid)

Triggers

Useful knowledge
Not tested on exams

- Trigger : a procedure that starts automatically if specified changes occur to the DBMS
- Active Database: a database with triggers
- A trigger has three parts:
  1. Event (activates the trigger)
  2. Condition (tests whether the trigger should run)
  3. Action (procedure executed when trigger runs)

- Database vendors did not wait for trigger standards! So trigger format depends on the DBMS
- NOTE: triggers may cause cascading effects.

Good way to shoot yourself in the foot

Triggers: Example (SQL:1999)

```
CREATE TRIGGER youngStudentUpdate
  AFTER INSERT ON Student
REFERENCING NEW TABLE NewStudent
FOR EACH STATEMENT
  INSERT INTO
     YoungStudent(snum, sname, major, standing, age)
     SELECT  snum, sname, major, standing, age
     FROM     NewStudent N
     WHERE  N.age <= 18;
```

event

newly inserted tuples

apply once per statement

action

Can be either before or after

That's nice. But how do we code with SQL?

- Direct SQL is rarely used: usually, SQL is embedded in some application code.
- We need some method to reference SQL statements.
- But: there is an *impedance mismatch* problem.
  - Structures in databases <> structures in programming languages
- Many things can be explained with the impedance mismatch.

The Impedance Mismatch Problem

The host language manipulates variables, values, pointers SQL manipulates relations.

There is no construct in the host language for manipulating relations. See https://en.wikipedia.org/wiki/Object-relational_impedance_mismatch

Why not use only one language?

- Forgetting SQL: “we can quickly dispense with this idea” [Ullman & Widom, pg. 363].
- SQL cannot do everything that the host language can do.

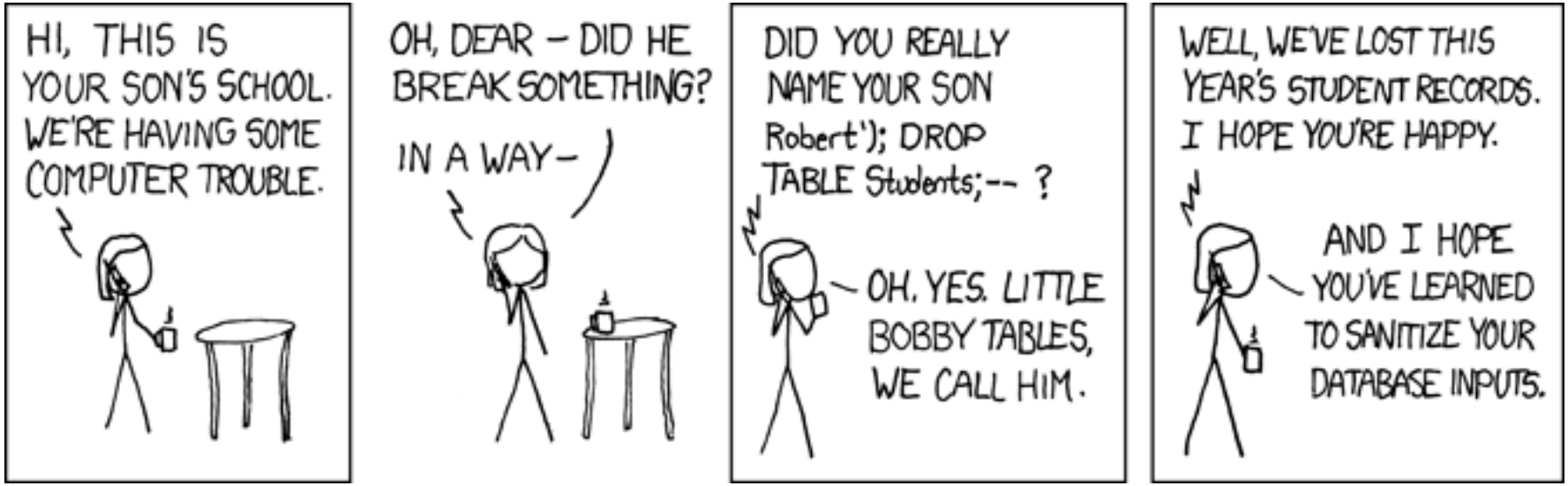
Database APIs

Rather than modify compiler, add library with database calls (API)

- Special standardized interface: procedures/objects
- Passes SQL strings from language, presents result sets in a language-friendly way – solves that impedance mismatch
- Microsoft's *ODBC* is a C/C++ standard on Windows
- Sun's *JDBC* a Java equivalent
- API's are DBMS-neutral
  - a "driver" traps the calls and translates them into DBMS-specific code

And now a brief digression

Have you ever wondered why some websites don't allow special characters?



Summary

- SQL was an important factor in the early acceptance of the relational model; more natural than earlier, procedural query languages.
- Relationally complete; in fact, significantly more expressive power than relational algebra.
- Consists of a data definition, data manipulation and query language.
- Many alternative ways to write a query; optimizer should look for most efficient evaluation plan.
  - In practice, users need to be aware of how queries are optimized and evaluated for best results.

Summary (Cont')

- NULL for unknown field values brings many complications
- SQL allows specification of rich integrity constraints (and triggers)
- Embedded SQL allows execution within a host language; cursor mechanism allows retrieval of one record at a time
- APIs such as ODBC and JDBC introduce a layer of abstraction between application and DBMS

Learning Goals Revisited

- Define the components (and synonyms) of the relational model: tables, rows, columns, keys, associations, etc.
- Create tables, including the attributes, keys, and field lengths, using Data Definition Language (DDL)
- Given the schemas of a relation, create SQL queries using: SELECT, FROM, WHERE, EXISTS, NOT EXISTS, UNIQUE, NOT UNIQUE, ANY, ALL, DISTINCT, GROUP BY and HAVING.
- Show that there are alternative ways of coding SQL queries to yield the same result. Determine whether or not two SQL queries are equivalent.
- Given a SQL query and table schemas and instances, compute the query result.
- Translate a query between SQL and English.
- Explain the purpose of NULL values and justify their use. Also describe the difficulties added by having nulls.
- Create and modify table schemas and views in SQL.
- Explain the role and advantages of embedding SQL in application programs.
- Write SQL for a small-to-medium sized programming application that requires database access.
- Identify the pros and cons of using general table constraints (e.g., CONSTRAINT, CHECK) and triggers in databases.